# LAPORAN PRAKTIKUM
# SISTEM OPERASI

## Modul 10 – Penjadwalan CPU 1





| | |
|---|---|
| NAMA | : Rosyid Al Ansorri |
| NIM | : L0125065 |
| KELAS | : A |
| HARI, TANGGAL | : Kamis, 5 November 2026 |
| WAKTU | : 08.20 - 09.10 WIB |
| DOSEN | : Ery Permana Yudha, S.Kom., M.Kom. |
| ASISTEN | : Benedhictus Kevin Doni (L0124006) |
| | Daffa Arrivo Aryanto (L0124093) |

**PROGRAM STUDI S1 INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA**

**UNIVERSITAS SEBELAS MARET**

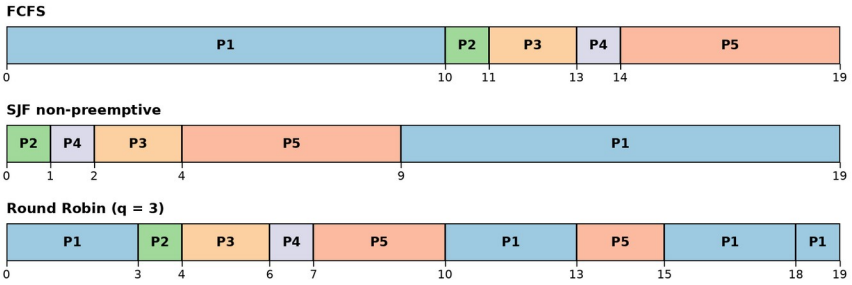

**2026**

# BAB I
# LANGKAH DAN PEMBAHASAN

## KEGIATAN 1

### Dokumentasi Kegiatan 1

1. Membuat Gantt Chart untuk masing-masing algoritma (FCFS, SJF non-preemptive, Round Robin q = 3)




2. Menghitung waiting time setiap proses dan waiting time rata-rata

**FCFS**

| Proses | BT | CT | TAT = CT - AT | WT = TAT - BT |
|---|---|---|---|---|
| P1 | 10 | 10 | 10 | 0 |
| P2 | 1 | 11 | 11 | 10 |
| P3 | 2 | 13 | 13 | 11 |
| P4 | 1 | 14 | 14 | 13 |
| P5 | 5 | 19 | 19 | 14 |

**AWT = (0 + 10 + 11 + 13 + 14) / 5 = 48 / 5 = 9,6 satuan waktu**

**SJF non-preemptive**

| Proses | BT | CT | TAT = CT - AT | WT = TAT - BT |
|---|---|---|---|---|
| P1 | 10 | 19 | 19 | 9 |
| P2 | 1 | 1 | 1 | 0 |
| P3 | 2 | 4 | 4 | 2 |
| P4 | 1 | 2 | 2 | 1 |
| P5 | 5 | 9 | 9 | 4 |

**AWT = (9 + 0 + 2 + 1 + 4) / 5 = 16 / 5 = 3,2 satuan waktu**

**Round Robin (q = 3)**

| Proses | BT | CT | TAT = CT - AT | WT = TAT - BT |
|---|---|---|---|---|
| P1 | 10 | 19 | 19 | 9 |
| P2 | 1 | 4 | 4 | 3 |
| P3 | 2 | 6 | 6 | 4 |
| P4 | 1 | 7 | 7 | 6 |
| P5 | 5 | 15 | 15 | 10 |

**AWT = (9 + 3 + 4 + 6 + 10) / 5 = 32 / 5 = 6,4 satuan waktu**

3. Catat parameter hasil eksekusi

| Algoritma | Urutan Eksekusi | Average Waiting Time |
|---|---|---|
| **FCFS** | P1 → P2 → P3 → P4 → P5 | 9,6 |
| **SJF non-preemptive** | P2 → P4 → P3 → P5 → P1 | 3,2 |
| **Round Robin (q = 3)** | P1 → P2 → P3 → P4 → P5 → P1 → P5 → P1 → P1 | 6,4 |

## A. Jawaban Analisis Kegiatan 1

1. Algoritma yang menghasilkan rata-rata waiting time paling rendah adalah SJF non-preemptive, yaitu 3,2 satuan waktu. Round Robin (q = 3) menghasilkan 6,4 dan FCFS menghasilkan 9,6. SJF unggul karena proses-proses yang pendek (P2, P4, P3, dan P5) dijalankan lebih dahulu, sehingga waktu tunggu total semua proses menjadi kecil.
2. FCFS dapat menyebabkan convoy effect karena prosesnya dijalankan menurut urutan kedatangan dan tidak dapat disela (non-preemptive). Pada data ini P1 yang berburst 10 berada di depan, sehingga P2, P3, P4, dan P5 yang pendek harus menunggu P1 selesai. Waiting time P2 sampai P5 menjadi 10, 11, 13, dan 14 satuan waktu padahal burst mereka hanya 1 sampai 5, dan akibatnya rata-rata waiting time FCFS menjadi tinggi.
3. Time quantum menentukan seberapa sering proses berganti. Quantum yang kecil membuat pembagian CPU lebih adil dan respons proses pendek lebih cepat, tetapi pergantian proses (context switch) menjadi sering sehingga overhead naik. Quantum yang besar membuat jumlah pergantian turun, dan jika quantum lebih besar atau sama dengan burst terpanjang (pada data ini $q \geq 10$), Round Robin berperilaku seperti FCFS dengan rata-rata waiting time 9,6. Pada q = 3, P2, P3, dan P4 selesai lebih awal dibanding FCFS, dan rata-rata waiting time turun menjadi 6,4.

## KEGIATAN 2

### A. Dokumentasi Kegiatan 2

1. Membuat direktori kerja dan file program fcfs.c

```
user@linux: ~/praktikum_cpu1
user@linux:~$ mkdir -p praktikum_cpu1
user@linux:~$ cd praktikum_cpu1
user@linux:~/praktikum_cpu1$ nano fcfs.c
```

```
user@linux: ~/praktikum_cpu1
  GNU nano 7.2                        fcfs.c
#include <stdio.h>
#include <string.h>

int main(void) {
    int n, at[100], bt[100], wt[100];
    int tat[100], ct[100], start[100];
    int i, j, temp, time = 0;
    int totalWT = 0, totalTAT = 0;
    char name[100][20], tempName[20];

    printf("=== Simulasi Penjadwalan CPU FCFS ===\n");
    printf("Jumlah proses (1-100): ");

    if (scanf("%d", &n) != 1 || n < 1 || n > 100) {
        printf("Jumlah proses tidak valid.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        printf("\nProses ke-%d\n", i + 1);

        printf("Nama proses: ");
        scanf("%19s", name[i]);

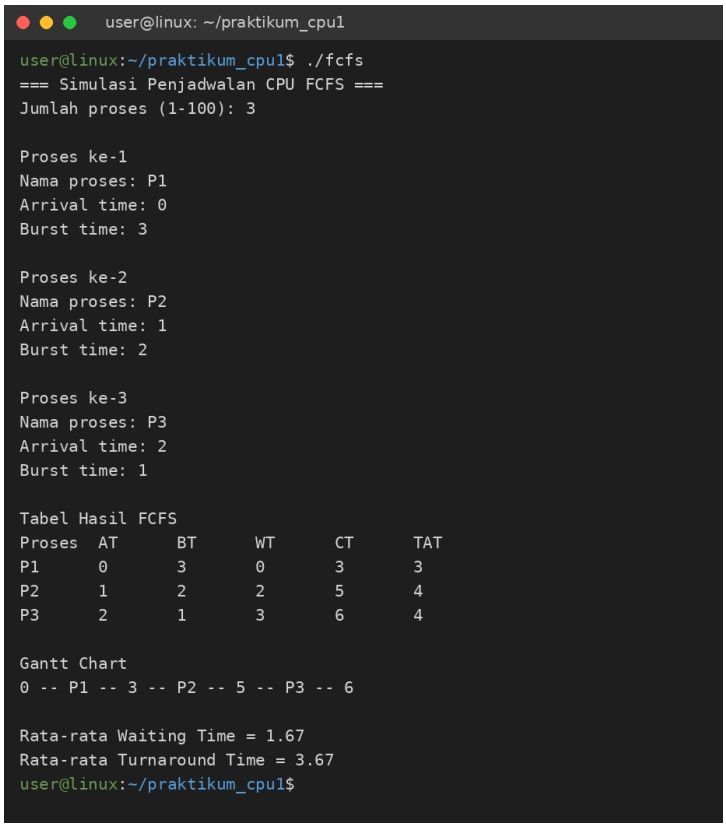
        printf("Arrival time: ");
        if (scanf("%d", &at[i]) != 1 || at[i] < 0) {
            printf("Arrival time tidak valid.\n");
            return 1;
        }

        printf("Burst time: ");
        if (scanf("%d", &bt[i]) != 1 || bt[i] <= 0) {
            printf("Burst time tidak valid.\n");
            return 1;
^G Help     ^O Write Out  ^W Where Is  ^K Cut
^X Exit     ^R Read File  ^\ Replace   ^U Paste
```

2. Mengompilasi program dengan gcc

```
user@linux: ~/praktikum_cpu1
user@linux:~/praktikum_cpu1$ gcc -Wall -Wextra fcfs.c -o fcfs
user@linux:~/praktikum_cpu1$ ls -l
total 20
-rwxr-xr-x 1 root root 16184 Nov  5 02:05 fcfs
-rw-r--r-- 1 root root  2446 Nov  5 02:05 fcfs.c
user@linux:~/praktikum_cpu1$
```

3. Menjalankan program simulasi FCFS dan memasukkan data pengujian




4. Catat parameter hasil eksekusi

| Aspek Pengamatan | Hasil Eksekusi |
|---|---|
| Urutan eksekusi proses | P1 → P2 → P3 |
| Waiting time P1 | 0 |
| Waiting time P2 | 2 |
| Waiting time P3 | 3 |
| Turnaround time P1 | 3 |
| Turnaround time P2 | 4 |
| Turnaround time P3 | 4 |
| Average Waiting Time | 1,67 |
| Average Turnaround Time | 3,67 |

## B. Jawaban Analisis Kegiatan 2

1. P2 datang pada waktu 1, saat P1 masih berjalan (P1 berjalan pada waktu 0 sampai 3). FCFS bersifat non-preemptive, yaitu proses yang sedang

memakai CPU tidak boleh disela sampai selesai. Karena itu P2 harus menunggu di ready queue sampai P1 selesai pada waktu 3, sehingga waiting time P2 = 3 − 1 = 2.

2. Arrival time (AT) dipakai program untuk mengurutkan proses menurut kedatangannya dan untuk menentukan waktu mulai (start = maks(waktu CPU saat ini, AT)), sehingga waiting time dihitung sebagai start − AT. Burst time (BT) menyatakan lama CPU yang dibutuhkan proses, dan dipakai untuk menghitung completion time (CT = start + BT) serta menentukan kapan proses berikutnya mulai berjalan.
3. Urutan eksekusi FCFS ditentukan oleh arrival time dari yang terkecil ke yang terbesar. Program mengurutkan data dengan bubble sort berdasarkan arrival time, lalu menjalankan proses dari indeks pertama sampai terakhir. Jika ada proses dengan arrival time sama, urutan input dipertahankan karena penukaran hanya terjadi bila at[j] > at[j+1]. Jika CPU kosong sebelum proses berikutnya datang, program mencatat waktu Idle.

## BAB II
## KESIMPULAN

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma penjadwalan CPU memengaruhi waiting time proses secara langsung. Pada lima proses dengan arrival time 0 dan burst time 10, 1, 2, 1, dan 5, SJF non-preemptive menghasilkan rata-rata waiting time terendah (3,2), disusul Round Robin dengan q = 3 (6,4), sedangkan FCFS paling tinggi (9,6) karena convoy effect. Simulasi FCFS dalam bahasa C berhasil dikompilasi dan dijalankan di Linux. Untuk tiga proses uji (P1, P2, P3), program menghasilkan urutan P1 → P2 → P3 dengan rata-rata waiting time 1,67 dan rata-rata turnaround time 3,67, yang sama dengan hasil perhitungan manual.